



MASTG

WebViews Cleanup

- MASTG-TEST-0250: References to Content Provider Access in WebViews
- MASTG-TEST-0251: Runtime Use of Content Provider Access APIs in WebViews
- MASTG-TEST-0252: References to Local File Access in WebViews
- MASTG-TEST-0253: Runtime Use of Local File Access APIs in WebViews
- MASTG-TEST-0258: References to Keyboard Caching Attributes in UI Elements

- L1
- L2
- ANDROID
- MASWE-0069
- TEST



MASTG-TEST-0252: References to Local File Access in WebViews

Overview

This test checks for references to methods from the `WebSettings` class used by Android WebViews which enable loading content from various sources, including local files. If improperly configured, these methods can introduce security risks such as unauthorized file access and data exfiltration. These methods are:

- `setAllowFileAccess` : allows the WebView to load local files from the app's internal storage or external storage.
- `setAllowFileAccessFromFileURLs` : lets JavaScript within those local files access other local files.
- `setAllowUniversalAccessFromFileURLs` : removes any cross-origin restrictions, allowing that JavaScript to read data across origins. The JavaScript **can always send data to any origin** (e.g. via `POST`), regardless of this setting; this setting only affects reading data (e.g. the code wouldn't get a response to a `POST` request but the data would still be sent).

When these settings are combined, they can enable an attack in which a malicious HTML file gains elevated privileges, accesses local resources, and exfiltrates data over the network, effectively bypassing the security boundaries typically enforced by the same-origin policy.

Even though these methods have secure defaults and are **deprecated in Android 10 (API level 29) and later**, they can still be explicitly set to `true` or their insecure defaults may be used in apps that run on older versions of Android (due to their `minSdkVersion`).

Refer to [Android WebView Local File Access Settings](#) for more information on these methods (default values, deprecation status, security implications), the specific files that can be accessed, and the conditions under which they can be accessed.

Example Attack Scenario:

Suppose a banking app uses a WebView to display dynamic content, and the developers enabled all three insecure settings. Additionally, JavaScript is enabled in the WebView.

- An attacker injects a malicious HTML file into the device (via phishing or another exploit) into a location that the attacker *knows* the WebView will access it from (e.g. thanks to reverse engineering). For example, an HTML file used to display the app's terms and conditions.
- The WebView can load the malicious file because of `setAllowFileAccess(true)`.
- Thanks to `setJavaScriptEnabled(true)` and `setAllowFileAccessFromFileURLs(true)`, the JavaScript in the malicious file (running in a `file:///` context) is able to access other local files using `file:///` URLs.
- The attacker-controlled script exfiltrates sensitive data from the device to an external server.

Note 1: Either `setAllowFileAccessFromFileURLs` or `setAllowUniversalAccessFromFileURLs` must be set to `true` for the attack to work. If both settings are set to `false`, the following error will appear in `logcat`:

```
[INFO:CONSOLE(0)] "Access to XMLHttpRequest at 'file:///data/data/org.owasp.mastestapp/files/api-key.tx"
[INFO:CONSOLE(31)] "File content sent successfully.", source: file:/// (31)
```

And the server would not receive the file content:

```
[*] Received POST data from 127.0.0.1:

Error reading file: 0
```

Note 2: As indicated in the Android docs, the value of `setAllowFileAccessFromFileURLs` is ignored if `allowUniversalAccessFromFileURLs=true`.

Steps

- Determine the `minSdkVersion` of the app.
- Use a tool like `semgrep` to search for references to:
 - the `WebView` class.
 - the `WebSettings` class.
 - the `setJavaScriptEnabled` method.
 - the `setAllowFileAccess`, `setAllowFileAccessFromFileURLs`, and `setAllowUniversalAccessFromFileURLs` methods from the `WebSettings` class.

Note that in this case **the lack of references to the `setAllow*` methods is especially interesting** and must be captured, because it could mean that the app is using the default values, which in some scenarios are insecure. For this reason, it's highly recommended to try to identify every `WebView` instance in the app.

Observation

The output should contain a list of `WebView` instances where the abovementioned methods are used.

Evaluation

The evaluation of this test is based on the [API behavior across different Android versions](#).

Fail:

The test fails if:

- `setJavaScriptEnabled` is explicitly set to `true`.
- `setAllowFileAccess` is explicitly set to `true` (or not used at all when `minSdkVersion` < 30, inheriting the default value, `true`).
- Either `setAllowFileAccessFromFileURLs` or `setAllowUniversalAccessFromFileURLs` is explicitly set to `true` (or not used at all when `minSdkVersion` < 16, inheriting the default value, `true`).

Pass:

The test passes if:

- `setJavaScriptEnabled` is explicitly set to `false`.
- `setAllowFileAccess` is explicitly set to `false` (or not used at all when `minSdkVersion` >= 30, inheriting the default value, `false`).
- Both `setAllowFileAccessFromFileURLs` and `setAllowUniversalAccessFromFileURLs` are explicitly set to `false` (or not used at all when `minSdkVersion` >= 16, inheriting the default value, `false`).

Best Practices

- MASTG-BEST-0010: Use Up-to-Date `minSdkVersion`
- MASTG-BEST-0011: Securely Load File Content in a `WebView`
- MASTG-BEST-0012: Disable JavaScript in `WebViews`

Demos

- MASTG-DEMO-0032: Uses of `WebViews` Allowing Local File Access with `semgrep`

